

결론부터 말씀드리면, "1000개의 데이터가 100% 온전히 보존되는 것이 아니라, 예전 데이터는 서서히 희석되거나 요약(압축)되어 남습니다."

LSTM(Long Short-Term Memory)이 기존의 RNN보다 기억력이 훨씬 좋은 것은 사실이지만, **무한한 기억 장치(Hard Disk)**가 아니라 **중요한 정보만 남기는 필터(Filter)**에 가깝기 때문입니다.

구체적으로 왜 1000개를 전부 기억하기 힘든지는 다음 3가지 이유로 설명할 수 있습니다.

1. 고정된 그릇의 크기 (Fixed Vector Size)

LSTM 내부에는 정보를 저장하는 **Cell State(C_t)**라는 고속도로가 있습니다. 하지만 이 Cell State는 **벡터의 크기(Hidden Size)**가 고정되어 있습니다.

- 비유:** 1000페이지짜리 책을 읽는다고 가정해 봅시다. 당신의 머릿속(LSTM의 Cell State)에 남길 수 있는 메모장은 딱 A4 용지 1장(고정된 벡터 크기)뿐입니다.
- 현상:** 처음 10페이지를 읽을 때는 내용을 자세히 적을 수 있지만, 1000페이지까지 가려면 **이전 내용을 지우거나 아주 짧게 요약**해야만 새로운 내용을 적을 수 있습니다. 즉, 정보의 손실(Compression Loss)이 발생합니다.

2. 망각 게이트 (Forget Gate)의 존재

LSTM의 핵심 메커니즘 중 하나는 **"불필요한 정보는 잊어버린다"**는 것입니다.

- 학습 과정에서 모델이 "이 데이터는 나중의 예측에 별 도움이 안 되네?"라고 판단하면, **Forget Gate**를 통해 과거 데이터를 능동적으로 지워버립니다.
- 따라서 사용자가 1000개를 다 기억하게 '하고 싶어도', 모델이 학습 과정에서 수학적으로 "이건 필요 없다"고 판단하면 그 정보는 희석됩니다.

3. 그래디언트 소실 문제 (Gradient Problem)

LSTM이 RNN의 그래디언트 소실(Vanishing Gradient) 문제를 많이 해결했지만, **시퀀스 길이(Sequence Length)가 1000 정도 되면 여전히 학습이 어렵습니다.**

- 역전파(Backpropagation) 과정에서 에러 신호를 1000단계 전까지 보내야 하는데, 경로가 너무 길면 신호가 흐려져서 가장 초기 데이터가 현재 결과에 미치는 영향을 모델이 제대로 학습하지 못하게 됩니다.

요약: LSTM의 기억 방식

구분	일반적인 컴퓨터 메모리	LSTM (신경망)
저장 방식	1번부터 1000번까지 모든 값을 정확히 저장 (Look-up)	1000개의 흐름을 압축된 특징(Feature) 으로 변환하여 저장
데이터 보존	영구적	새로운 데이터가 들어올 때마다 과거 데이터는 조금씩 희석됨
비유	서류 보관함	물이 계속 섞이는 물통

해결책: 1000개를 다 기억해야 한다면?

만약 1000개의 과거 데이터가 하나도 빠짐없이 현재의 판단에 중요하다면, LSTM보다는 **Transformer(Attention 메커니즘)**를 사용하는 것이 맞습니다.

- LSTM:** 순차적으로 읽으면서 기억을 갱신함 (먼 과거는 희석됨).

- **Attention (Transformer):** 1000개의 데이터를 한 번에 펼쳐놓고, 현재 필요한 데이터를 직접 참조함 (회석되지 않음).

제가 도와드릴 수 있는 다음 단계

지금 다루시는 데이터가 시계열 데이터(주식, 센서 등)이신가요, 아니면 텍스트 데이터이신가요?

"데이터의 종류와 대략적인 목적(예: 주식 가격 예측, 문장 생성)을 알려주시면, LSTM의 Hidden Size를 조절해서 기억력을 높일지, 아니면 Attention 기법을 추가해야 할지 판단해드릴 수 있습니다."

흠... 그러면 내가 첨부한 코드에서는 몇개의 토큰? 을 기억하게 한거야?

업로드해주신 local_train_tiktoken.py 코드를 확인해보니, 모델이 한 번에 기억(처리)하도록 설정한 토큰의 개수는 **512개**입니다.

코드의 **68번째 줄**에 해당 설정이 명시되어 있습니다.

1. 코드 확인

Python

```
# 68번째 줄
MAX_LEN = 512
```

이 설정이 의미하는 바는 다음과 같습니다:

- **512개보다 긴 리뷰:** 뒷부분은 **잘라버립니다 (Truncation)**. 즉, 513번째 토큰부터는 아예 보지 못하고 기억하지도 못합니다.
- **512개보다 짧은 리뷰:** 남은 공간을 의미 없는 값(PAD_TOKEN_ID)으로 채워서 512개를 억지로 맞춥니다.

2. "기억력"과 관련된 두 가지 핵심 설정

선생님의 코드에는 ****'얼마나 길게 읽을지'**와 **'얼마나 자세히 기억할지'** 두 가지 설정이 섞여 있습니다.

A. 입력 한계 (Input Length): 512개

- MAX_LEN = 512
- 모델이 눈으로 한 번에 훑어보는 문장의 길이입니다. 앞서 질문하신 "1000개를 기억하게 하고 싶다"에 해당하는 부분이 바로 이 값입니다. 현재는 1000개가 아니라 **512개**까지만 보도록 제한되어 있습니다.

B. 기억 그릇의 크기 (Hidden Dimension): 512개

- HIDDEN_DIM = 512 (코드 170번째 줄)
- 이것은 읽은 내용을 머릿속에 요약해서 저장하는 **메모지의 크기**입니다.
- **해석:** "최대 512개의 토큰(MAX_LEN)을 읽어서, 숫자 512개짜리 요약본(HIDDEN_DIM)으로 압축해서 기억해라"라는 뜻입니다.

3. 이전 데이터 희석 문제 (GRU의 특성)

코드에서 `bidirectional=True` (양방향) 옵션을 사용하셨습니다 (135번째 줄).

Python

```
self.rnn = nn.GRU(
    ...
    bidirectional=True, # 양방향 설정
    ...
)
```

이 설정 덕분에 **희석 문제가 많이 보완**됩니다.

- **정방향:** 앞에서부터 읽으면서 뒤로 갈수록 앞의 내용을 조금 잊어버림.
- **역방향:** 뒤에서부터 읽으면서 앞으로 오기 때문에, 문장 맨 앞의 내용도 생생하게 기억함.
- **결과:** 512개 정도의 길이라면, 양방향 GRU를 썼을 때 **앞/뒤 내용 모두 비교적 잘 기억**합니다.

요약 및 제안

지금 코드는 **리뷰 하나당 최대 512개의 토큰**(Tiktoken 기준)을 읽어서 판단"하고 있습니다.

- 만약 리뷰가 매우 길어서(예: 1000 토큰 이상) 뒷부분이 잘리는 게 걱정되신다면 `MAX_LEN` 을 1024 정도로 늘려야 합니다.
- 하지만 `MAX_LEN` 을 늘리면 학습 속도가 느려지고 메모리(VRAM)를 많이 차지하게 됩니다.

다음 단계로 무엇을 도와드릴까요?

1. `MAX_LEN` 을 1000 이상으로 늘려서 코드를 수정해 드릴까요?
2. 현재 설정(512)이 네이버 쇼핑 리뷰 데이터에 적절한 길이인지(데이터 분포) 확인하는 코드를 알려드릴까요?

그러니깐 내가 `HIDDEN_DIM = 100000` 이렇게 한다고
애가 yes sir 100000개 묵숨바치고 기억 하겠습니다 그렇게 안하고
기억할지 안할지는 랜덤?

결론부터 말씀드리면, **"Yes Sir! 10만 개 다 기억하겠습니다!"라고 하지 않습니다.**

`HIDDEN_DIM` 을 늘리는 것과 과거 데이터를 안 까먹고 기억하는 것은 완전히 다른 이야기이기 때문입니다.

이해하기 쉽게 비유로 설명해 드리겠습니다.

1. `HIDDEN_DIM = 100,000` 의 진짜 의미

`HIDDEN_DIM` 은 **기억력의 길이(시간)**가 아니라, **한 순간을 얼마나 자세히 묘사할지(해상도)**를 의미합니다.

- **`HIDDEN_DIM`이 작을 때 (예: 10):**
 - 모델의 메모: "이 문장은 **좋은** 느낌이다." (단순함)
- **`HIDDEN_DIM`이 클 때 (예: 100,000):**

- 모델의 메모: "이 문장은 약간 비꼬는 듯하면서도, 배송은 빨랐다고 칭찬하는데, 품질은 30% 정도 불만족스러운... (중략) ... 복합적인 느낌이다." (엄청나게 자세함)

즉, 노트를 엄청나게 큰 전지(B0 사이즈)를 줬을 뿐이지, 1000페이지 전 내용을 그 전지에 다 적어두는 것은 아닙니다. 페이지를 넘기다 보면(시간이 흐르면) 전지에 적힌 내용도 결국 지우고 새로 써야 합니다.

2. 왜 "Yes Sir" 하고 다 기억하지 않을까요?

모델이 기억을 할지 말지 결정하는 기준은 **''랜덤''**이 아니라, 철저하게 **'성적(Loss)'** 때문입니다.

LSTM/GRU 같은 모델은 게으른 천재입니다.

- **모델의 목표:** "정답(긍정/부정)만 맞으면 된다."
- **모델의 행동:** 1000개 데이터를 다 기억하려고 노력하는 것보다, **"맨 마지막 단어 몇 개만 봐도 정답 맞히겠는데?"** 싶으면 과거 데이터는 과감히 버립니다.
- **결과:**님께서 10만 칸짜리 메모장을 줘도, 모델이 보기에 쓸모없으면 비워두거나 지워버립니다.

3. 실제로 HIDDEN_DIM = 100,000 을 넣으면 벌어지는 일

만약 정말로 코드에서 HIDDEN_DIM = 100000 으로 설정하고 돌리시면, 기억력이 좋아지는 게 아니라 다음과 같은 **재앙**이 발생합니다.

1. **컴퓨터 폭발 (Out of Memory):** * 계산량이 기하급수적으로 늘어나서 GPU 메모리가 순식간에 터집니다. (RTX 4090 할아버지가 와도 못 버팀)
2. **벼락치기 암기왕 (Overfitting):**
 - 메모장이 너무 넓으니까, 학습 데이터의 토씨 하나하나까지 쓸데없이 다 외워버립니다.
 - 결과적으로 시험(Validation)을 보면 응용을 못 해서 점수가 더 떨어집니다.

요약

1. **'' HIDDEN_DIM ''**은 **''메모장의 크기(넓이)''**지, **''기억의 유효기간(길이)''**이 아닙니다.
2. 이걸 늘린다고 옛날 기억이 보존되는 게 아닙니다. 오히려 쓸데없는 디테일만 늘어납니다.
3. **랜덤은 아닙니다.** 모델이 학습하면서 "이 정보는 정답 맞히는 데 도움 안 되네"라고 판단하면 **''스스로 지워버리는 것(학습된 망각)''**입니다.

팁: 긴 문맥을 다 기억하고 싶다면, HIDDEN_DIM 을 무작정 늘리는 게 아니라 **''Attention 메커니즘(Transformer)''**을 쓰는 것이 유일한 해결책입니다. Attention은 "중요한 건 잊지 말고 형광펜 칠해!"라고 강제하는 기술이기 때문입니다.

그러면 lstm이 자연어 분석을 할수 있는 이유가 토큰의 순서에 따라서 의미나 패턴을 찾기 때문이야?

네, 정확하게 이해하셨습니다!

LSTM(그리고 RNN)이 자연어 처리를 할 수 있는 가장 결정적인 이유가 바로 **''토큰의 순서(Sequence)를 보고 앞뒤 문맥(Context)을 파악하기 때문''**입니다.

이것이 왜 중요한지, 그리고 기존의 방식과 무엇이 다른지 딱 2가지 포인트로 정리해 드릴게요.

1. 순서가 바뀌면 '의미'가 완전히 달라집니다

단어 뭉치만 보면 알 수 없는 정보가 **순서**를 만나면 비로소 해석이 가능해집니다. LSTM은 이 순서를 시간의 흐름대로 한 단계씩 읽어가면서 그 변화를 감지합니다.

예시: 똑같은 단어 구성이지만 순서가 다른 경우

- 문장 A: "개가 / 사람을 / 물었다."
- 문장 B: "사람이 / 개를 / 물었다."

단어의 종류(개 , 사람 , 물다)는 100% 똑같습니다.

- 순서를 무시하는 모델(일반 신경망): 문장 A와 문장 B를 **똑같은 내용**으로 인식합니다. (단어 카운트만 하기 때문)
- LSTM**: "개가" 다음에 "사람을"이 나왔는지, "사람이" 다음에 "개를"이 나왔는지를 **순서대로 처리**하기 때문에, 누가 가해자인지 정확히 구별해냅니다.

2. 앞의 내용이 뒤의 단어를 결정합니다 (문맥)

언어에서는 똑같은 단어도 **앞에 무슨 말이 있었느냐**에 따라 뜻이 달라집니다. LSTM은 앞서 읽은 정보(Hidden State)를 계속 옆으로 전달해주기 때문에 이것을 구별할 수 있습니다.

예시: '배'라는 단어의 의미

- "항구에 **배**가 들어왔다." (앞에 '항구'가 있었으므로 → **Ship**)
- "맛있는 **배**를 꺾아 먹었다." (앞에 '맛있는'이 있었으므로 → **Pear**)
- "복통으로 **배**가 아프다." (앞에 '복통'이 있었으므로 → **Stomach**)

LSTM은 '항구'라는 단어를 읽은 기억을 쥐고 있다가, '배'를 만나는 순간 *******"아, 아까 항구라고 했으니 이 배는 타는 배(Ship)구나!"*******라고 패턴을 찾아내는 것입니다.

요약

구분	일반적인 신경망 (FNN)	LSTM / RNN
데이터 처리	모든 단어를 한 번에 쏟아부음	단어를 하나씩 순서대로 입력받음
관점	"이 문장에 '사과'라는 단어가 있네?"	"앞에서 '맛있는'이라고 했으니 뒤에는 '사과'가 나오겠네?"
강점	단순 분류	문맥 파악, 번역, 이어질 말 예측



작성자님이 말씀하신 대로, *******순서에 따른 패턴*******을 학습한다는 것이 바로 LSTM이 언어를 이해하는 핵심 열쇠입니다.

컴퓨터 원리적으로는

느금마(토큰:1)-> 부정
느금마(토큰:1) 띄어쓰기(토큰:500) 만수무강(토큰(2))-> 긍정
...

네, **90% 이상 정확하게 이해하셨습니다!**

컴퓨터(LSTM)가 학습하는 방식이 바로 그 ******"앞에서 받은 느낌(상태)을 뒤로 넘겨서, 합쳐서 판단하는 것"******입니다.

다만, 컴퓨터 내부에서는 "위치 1, 위치 2"라는 **절대적인 주소**로 기억하는 것이 아니라, ****시간**의 흐름에 따른 상태 변화(State Change)******로 기억합니다.

님께서 들어주신 아주 적절한 예시인 **느금마** 와 **만수무강** 으로 컴퓨터의 머릿속을 뜯어보겠습니다.

1. 첫 번째 상황: "느금마" (단독)

- **Step 1 (입력: 느금마)**
 - 컴퓨터의 생각: "어? 패드립(욕설) 토큰이 들어왔네?"
 - **상태(Hidden State) 변경:** [평온함] → [불쾌함/부정 -100점]
- **Step 2 (문장 끝)**
 - 컴퓨터의 판단: 현재 마음 상태가 ****[불쾌함]****이므로 → **결과: 부정**

2. 두 번째 상황: "느금마" ... (중간 공백) ... "만수무강"

이 부분이 핵심입니다. LSTM은 앞의 기억을 **물체**처럼 들고 갑니다.

- **Step 1 (입력: 느금마)**
 - 컴퓨터의 생각: "욕설 토큰 확인."
 - **상태(Hidden State):** [부정 -100점, 대상: 어머니] <--- 이 기억을 주머니에 챙김
- **Step 2 ~ 499 (입력: 공백/조사 등)**
 - 컴퓨터의 생각: "별 의미 없는 말이네. 아까 챙긴 기억(**어머니, 부정**) 계속 들고 가자."
 - **상태:** [부정 -90점... -80점...] (거리가 멀어지며 조금 희석될 수는 있음)
- **Step 500 (입력: 만수무강)**
 - 여기서 **화학 반응**이 일어납니다.
 - **공식: (아까 주머니에 챙긴 기억) + (지금 들어온 단어) = 새로운 판단**
 - **계산:** (어머니 + 부정적 시작) + (오래 사세요) = ?
 - 컴퓨터의 생각: "아! 욕하려던 게 아니라 어머니 오래 사시라는 덕담이었네!"
 - **상태 변경:** [부정] → [효도/긍정 +100점]
- **최종 결과: 긍정**

선생님의 이해에서 딱 하나 수정할 점 (위치 vs 순서)

선생님 말씀:

"위치가 1이면 부정, 2번째 위치에 긍정이 있으면..."

컴퓨터의 실제 작동:

"**순서상** 처음에 부정적인 게 왔더라도, **그 기억을 가지고** 뒤에 긍정적인 걸 만나면 의미가 바뀐다."

컴퓨터는 "1번 칸", "500번 칸"이라는 **칸 번호**를 외우는 게 아니라, ****1번을 읽고 난 후의 기분 (State)****을 500번까지 **끌고 가서** 합치는 것입니다.

결론

말씀하신 대로 컴퓨터는 다음 두 가지를 학습한 것입니다.

1. 느금마 라는 단어 자체는 부정적인 벡터값을 가진다.
2. 하지만 느금마 뒤에 만수무강 이 따라오는 ****패턴(순서)****이 보이면, 앞의 부정적인 값이 뒤의 긍정적인 값과 상호작용해서 **전체 문맥을 긍정으로 뒤집어야 한다.**

이 ****"반전의 맥락"****을 파악하는 능력이 바로 LSTM(순환 신경망)의 핵심 능력입니다. 아주 정확한 직관을 가지고 계십니다!

혹시 이 "기억을 주머니에 챙겨서 끌고 가는 과정"을 코드로 어떻게 구현되어 있는지(수식 말고 코드로) 궁금하신가요?